Project Report: IPL Web Automation Testing

1.Project Overview

This project is a specialized, end-to-end automation suite engineered to validate the functional integrity, UI consistency, and data accuracy of a high-traffic IPL (Indian Premier League) web portal. By leveraging the combined power of **Selenium WebDriver** and **TestNG**, the suite ensures that dynamic elements—such as real-time team statistics, historical trophy records, and live-updating points tables remain accurate across diverse browser updates and UI refreshes. The project serves as a robust regression framework, significantly reducing manual testing overhead and minimizing the risk of human error in the verification of critical, fast- changing sports data.

2.Project Explanation

The primary objective of this project is to implement a resilient automation layer for the IPL website, where data frequently changes due to live match results and seasonal updates. The key functional areas include:

- **Advanced Dynamic Data Scrapping:** Verifying team trophies and winning years. This requires sophisticated interaction with hover-over elements that only appear in the DOM upon specific mouse events. We utilized precise coordinate-based triggers and event-driven automation to capture data that is otherwise hidden from standard "get-text" methods.
- **UI Integrity & Visual Consistency:** Ensuring that critical branding elements like logos, along with navigational components like header/footer links, are rendered correctly across various screen resolutions. This prevents "broken-link" scenarios and visual regressions that could degrade the professional user experience expected of a major sports brand.
- **Real-time Data Accuracy:** Cross-verifying the Points Table data (Rank, Matches, Points) against expected datasets. This ensures that the logic used to parse the complex HTML table structures remains valid even as the standings shift daily, providing a continuous verification of the portal's backend data synchronization.
- **Operational Robustness:** Implementing sophisticated handlers for asynchronous elements, such as cookie banners, third-party advertisements, and lazy-loading team cards. By addressing these "non-deterministic" UI components, we have created a suite that avoids the common pitfalls of "flaky" tests that often plague standard automation scripts.

3.Project Architecture

The project adheres to a **Modular Page Object Model (POM)** architecture, an industry- standard design pattern that strictly decouples the test logic from the underlying UI structure.This separation of concerns ensures that the suite is easy to maintain; if the UI changes, only the Page Objects need updates, leaving the test scripts untouched.

A. Architecture Diagram

```
[ Resource Layer ] <------ [ Suite Config (testng.xml) ]
          |                                  |
          v                                  v
[ Base Layer (BaseTest.java) ] --> [ Test Layer (Test Scripts) ]
          |                                  ^
          v                                  |
[ Page Layer (POM Classes) ]----------------------------/
(HP, TP, PP)
```

B.Component Hierarchy & Design Philosophy

1. **Base Layer (** BaseTest.java **):** This serves as the structural foundation of the framework. It manages the entire lifecycle of the WebDriver, including the configuration of browser- specific capabilities—such as headless execution for CI/CD pipelines, incognito mode to avoid cached data, and global implicit/explicit timeouts. It ensures a sanitized testing environment by executing vital teardown activities (driver.quit()) after every suite run to prevent memory leaks and orphaned browser processes.
2. **Page Layer (POM):** Utilizing the PageFactory pattern, classes like IPLHomePage.java and TeamPage.java encapsulate the web elements using @FindBy annotations. This layer acts as an abstraction barrier; it provides high-level action methods (e.g., searchForTeam() , getPointsTableData()) to the test layer, hiding the complex XPaths and CSS selectors required to find those elements.
3. **Test Layer:** This layer contains the core business logic and validation rules. By using TestNG annotations like @Test, @BeforeMethod, and @AfterMethod, the suite manages the execution flow with precision. It performs rigorous hard and soft assertions to validate that the actual UI state perfectly matches our expected data models.
4. **Resource Layer:** This provides centralized orchestration through testng.xml, which allows for parallel execution, test grouping, and parameterization. Meanwhile, the pom.xml file manages the project's dependency lifecycle, ensuring that all libraries including Selenium 4.x

features, TestNG, and WebDriverManager—are always synchronized across the team's development environments.

C. Advanced Design Patterns & Implementation Strategies

- **Singleton/Factory Pattern:** We implemented a custom Driver Factory to manage WebDriver instances. This prevents the unnecessary overhead of opening multiple browser windows for the same test class, significantly optimizing system resource consumption.
- **Fluent Wait/Explicit Strategy:** To move away from unreliable static sleeps (Thread.sleep), the framework utilizes WebDriverWait combined with ExpectedConditions . This allows the script to proceed the millisecond an element.
- **Action Class Orchestration:** This was essential for the IPL site's interactive card system. The Actions class is used to chain complex sequences, such as moving to a team card, waiting for the animation to complete, and then capturing the specific trophy icon's tooltip data to simulate a genuine human interaction

4. In-Depth Test Case Documentation & Output

This section details the specific technical challenges encountered during development and the strategic automation logic applied to overcome them.

TC_001: Footer Link Integrity Validation

- **Objective:** To verify the structural completeness and navigational health of the global footer across four primary categories: TEAM, ABOUT, GUIDELINES, and CONTACT.
- **Challenge:** The footer contains a high density of links (29 per section) that may be obscured by "back-to-top" buttons or dynamic sticky footers that overlap the bottom of the viewport.
- **Solution:** We implemented a collection-based approach using List<WebElement>. The script identifies the parent containers for each section and then retrieves all child <a> tags. By validating the size of these lists, we ensure that the site's structural integrity remains intact. This acts as a vital "smoke test" to ensure no major navigational regressions have occurred.
- **Execution Output:**

```
TEAM links count: 29
ABOUT links count: 29
GUIDELINES links count: 29
CONTACT links count: 29
```

TC_002: Dynamic Team Logo & Trophy Verification

- **Objective:** To validate the visual rendering of all 10 team identities and the accuracy of their historical trophy counts.
- **Challenge:** Winning years are hidden within hover-activated tooltips that do not exist in the DOM until the hover event is triggered. Additionally, the script must handle newer teams (like LSG or

DC) that currently have no trophies, ensuring the script doesn't crash when it fails to find a trophy icon.

- **Solution** : We developed a "Hover-and-Extract" logic. For each team card, the script performs a moveToElement() action. We then use a partial CSS selector [class*='trophy'] to capture the trophy icons dynamically. A conditional logic handler was integrated to gracefully log "NO_TITLES" for teams without wins, allowing the loop to continue seamlessly across all 10 teams.
- **Execution Output :**

```
Processing team: CHENNAI SUPER KINGS
CHENNAI SUPER KINGS - Logo: PASS
Found winning years with selector [class*='trophy']: 2010 | 2011 | 2018 | 2022
 CHENNAI SUPER KINGS - Winning years: PASS
...
Processing team: DELHI CAPITALS
DELHI CAPITALS - Logo: PASS
DELHI CAPITALS - Hover years
detected: NO_TITLES
DELHI CAPITALS - No titles: PASS
```

TC_003: Points Table Data Synchronization

- **Objective:** To ensure the live data grid accurately reflects the current league standings, focusing on the Rank 1 team
- **Challenge:** HTML tables are notoriously difficult to automate because row indices shift as teams win or lose matches. Hard-coding the row number would cause the test to fail the moment the standings change.
- **Solution:** We utilized relative XPaths and the following-sibling axis. By anchoring the search to the "Rank 1" text, the script dynamically finds the adjacent cells for the Team Name, Matches Played, and Total Points. This "anchor-and-offset" strategy ensures the test remains valid regardless of which team climbs to the top of the table.
- **Execution Output:**

```
Rank 1 Team: RR
Team Name check: PASS -> RR
Rank 1 Matches Played: 3 Matches
Played check: PASS -> 3 Rank 1
Points: 6
Points check: PASS -> 6
Points Table Test PASSED for team: RR
```

TC_004: Global Search Accuracy

- **Objective:** To test the search engine's indexing capabilities and ensure the UI correctly displays filtered results

- **Challenge:** The search overlay often triggers a global cookie consent banner that physically blocks the input field, leading to "ElementClickIntercepted" errors.
- **Solution:** We built a "Pre-Action" utility that checks for the visibility of the cookie banner before every search. If detected, the banner is closed; if not, the script proceeds via an exception-handling block. After inputting the search term, we use visibilityOfAllElementsLocatedBy to ensure the results (e.g., the 6 expected results) are fully rendered before we perform the final assertion.
- Execution Output:

```
No cookie popup
Search Results: 6
```

5. Team Roles & Partitioning

To maximize development efficiency and ensure comprehensive coverage, the workload was partitioned into two distinct technical tracks based on individual expertise:

A. Mupparaju Sai Ram

- **Focus Area:** Framework Backbone & Data Reliability.
- **Key Contributions:** Sai Ram was responsible for the architectural setup of the BaseTest class and the implementation of the complex Points Table synchronization logic (TC_003). He specialized in developing the robust, anchor-based XPath strategies required to extract data from highly dynamic HTML tables and ensured that the footer link validations were synchronized across multiple browser viewports.

B.Mahammad Reshma

- **Focus Area:** Interactive UI Components & Synchronization Utilities.
- **Key Contributions:** Reshma developed the interactive "Hover" logic for the Team Cards (TC_002) and designed the Global Search module (TC_004). Her work focused on the user-centric features of the site, implementing the Actions class workflows and the critical "Smart-Cookie" handler that prevents test interruptions from intrusive marketing pop-ups and overlays.

6.Technical Stack Summary

- **Language:** Java 11/17 — Leveraging Object-Oriented principles for scalable POM structures.
- **Automation Tool:** Selenium WebDriver (v4.x) — Utilizing advanced features like Relative Locators and improved window handling.
- **Test Framework:** TestNG (v7.11.0) — Chosen for its robust reporting, data-driven capabilities, and suite management.
- **Browser:** Chrome (v146) — Controlled via ChromeDriver for high-performance execution.
- **Logging:** SLF4J (Simple Logging Facade) — Used for standardized, readable console reporting and debugging.

7. Execution Summary

The suite was executed in a clean-room environment to ensure zero interference from local cache or cookies.

- **Total Tests Executed:** 4
- **Successful Passes:** 4
- **Failures/Skips:** 0
- **Total Execution Time:** ~68 seconds (including browser spin-up and network latency).
- **Status:** The suite is confirmed stable, repeatable, and ready for integration into a continuous integration (CI/CD) pipeline for daily health checks.

8. Conclusion

The successful completion of this project represents a significant milestone in creating a stable, automated verification environment for the IPL web portal. By utilizing the **Page Object Model** and proactively addressing common automation hurdles—such as dynamic hovers, asynchronous overlays, and shifting table data—the framework provides a reliable safety net for future regression testing. The close collaboration between **Mupparaju Sai Ram** and **Mahammad Reshma.**